Application graphique client/serveur Attribution de sujets

Cadre et objectif général

Le projet vise à développer une application graphique PyQt client/serveur qui permet d'attribuer automatiquement des sujets de projet en fonction des choix priorisés faits par les stagiaires.

Cette application comprend :

1 interface graphique administration qui permet de paramétrer l'application :

1. renseigner le titre et le descriptif de N sujets

2. indiquer le nombre P de personnes par sujet

3. indiquer le nombre M < N de choix de sujet par personne

4. indiquer la date limite pour faire le choix

1 interface graphique utilisateur qui permet :

1. de créer un compte : nom, prénom, login/mot de passe

2. de choisir et ordonner ses sujets préférés

1 interface d'administration qui permet de visualiser :

• pour chaque stagiaire : la liste des sujets et leur ordre de préférence

• des statistiques : sujets les plus et les moins choisis, nombre de choix moyen par personne

Après la date limite, attribution des premiers choix, puis des 2e choix, etc., en cascade. Si le nombre C de stagiaires ayant choisi un sujet est supérieur au nombre P de personnes par sujet, un tirage au sort de P stagiaires parmi C est effectué.

Chaque personne voit son classement dans la liste d'attente pour chaque projet.

• 1 interface d'administration pour visualiser à chaque étape de l'algorithme l'appariement sujet - P personnes, et le nombre de personnes en liste d'attente.

Les étapes principales du projet

Les étapes principales du projet consistent à :

• réfléchir aux données qui devront transiter du serveur vers les clients et vice-versa

• concevoir et créer une base de données qui permet de gérer les données

• proposer une architecture pour l'application client/serveur

• concevoir un serveur qui aura les fonctionnalités suivantes :

- création, suppression, modification des comptes des utilisateurs

- authentification des utilisateurs

- envoi des sujets et de leur description aux clients

- réception des vœux des clients

- calculs d'attribution des sujets

• concevoir et programmer le client

• dessiner les interfaces graphiques client concernant :

- l'authentification

- la création/modification/suppression de compte

- le choix des sujets

- la visualisation du sujet attribué

• programmer l'interface graphique

Environnement de développement et dépôt GIT

Le projet doit :

• utiliser l'outil de gestion de version Git et un IDE de développement Python

• être structuré suivant l'arborescence indiquée ci-après

• pouvoir s'exécuter sur le système Linux de la Machine Virtuelle deb12-lxqt du datacenter lors de la démonstration finale

• être documenté : description du projet au format restructuredText, commentaires pertinents dans le code (si utiles à la compréhension), commentaires des fonctions développées

• comporter un répertoire de test où toutes les fonctions Python développées auront un code de test unitaire

Le projet est rattaché à un dépôt GIT que vous aurez créé sur GitHub et à la livraison de vos codes informatiques. Le dépôt **devra absolument** être remis lors de la livraison finale du projet. Le versionnement étant tracé et daté, il servira pour l'évaluation du travail du groupe et de chaque étudiant.

Langages de scripting/programmation

Le projet doit utiliser le langage de programmation **Python (version > 3.10)** pour les **codes sources** et **PyQt** pour les projets d'interface graphique.

Arborescence du projet

Votre projet doit être exécuté par le biais d'un script **nom_projet.py**. Ce script reprendra la structure classique des programmes vue en **R1.07 - Fondamentaux de la programmation** et décrite dans le formulaire Python. Il prendra d'éventuels paramètres en arguments.

Il doit respecter l'arborescence suivante. **PROJETGitHUB** désigne le répertoire auquel est rattaché votre projet et constitue la base du dépôt local Git.

```
PROJETGitHUB/
├── .git/
├── data/
│   └── ...
├── docs/
│   ├── build/
│   │   └── html/
│   └── source/
│       ├── index.rst
│       ├── conf.py
│       ├── content/
│       ├── _static/
│       └── _templates/
├── html/
│   └── ...
├── __init__.py
├── nomprojet/
│   ├── nom_projet.py
│   └── nom_module_projet.py
├── tests/
│   ├── __init__.py
│   └── test_nom_module_projet.py
├── .gitignore
├── AUTHORS
└── requirements.txt
```

Attention et notes

• Le fichier **.gitignore** commence par un point.

• Vous pouvez ajouter au besoin autant de modules que nécessaire pour structurer votre code, en les stockant à la racine du répertoire **nomprojet**.

• Un modèle **Modèle Vue Contrôleur (MVC)**, avec l'aide MVC_help, est à privilégier pour structurer le code de l'application.

Rôle des répertoires et fichiers

• **.git** : répertoire dédié à Git.

• **data** : répertoire dédié à stocker différents fichiers de données récupérées et générées pour les besoins du projet.

• **docs** : répertoire dédié à stocker la documentation du projet au format restructuredText (répertoire généré automatiquement par sphinx-build).

• **html** : répertoire contenant le site web statique de présentation des résultats.

• **__init__.py** : fichier indiquant la version du projet :

```
__version__ = '0.1.0'
```

• **nomprojet** : répertoire dédié aux fichiers source Python développés lors du projet.

• **tests** : répertoire dédié aux tests unitaires des fonctions développées dans le projet.

• **tests/__init__.py** : fichier vide.

• **.gitignore** : fichier permettant de configurer Git pour ne pas envoyer sur le dépôt distant les fichiers temporaires.

• **AUTHORS** : fichier indiquant le nom des auteurs et leurs coordonnées.

• **requirements.txt** : fichier texte décrivant la version de Python utilisée et les dépendances du programme Python (modules et versions des modules Python).

Documentation

• La documentation générale du projet devra être écrite au format restructuredText. Vous pourrez pour cela vous appuyer sur le logiciel Sphinx.

• Il conviendra d'ajouter des commentaires **docstrings** en début de fonction afin de :

- préciser ce que fait la fonction

- indiquer son auteur, ses dates de création et de dernière modification

- décrire ses paramètres et, le cas échéant, leurs types

- décrire les bornes d'utilisation des paramètres pour un bon fonctionnement de la fonction et les exceptions susceptibles d'être levées

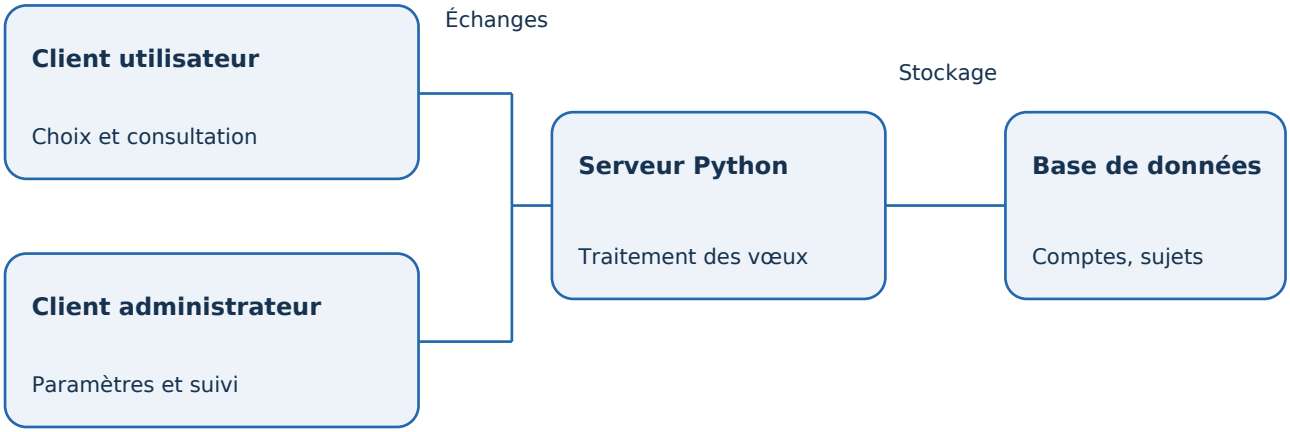
- indiquer ce qu'elle retourne

- donner un exemple d'utilisation

Source du texte : énoncé de l'Université de Poitiers. Texte fourni par l'utilisateur ; présentation et illustrations ajoutées.

Illustrations du projet

Schéma de principe et maquettes proposées pour illustrer le sujet. Les noms, dates et résultats affichés sont des exemples fictifs.



Le client est l'application ouverte par une personne. Le serveur reçoit les demandes et calcule les attributions. La base de données conserve les informations.

Exemple : interface utilisateur

Attribution de sujets - Espace utilisateur

Mes préférences Date limite : 20/10/2026

1er choix	Supervision réseau
2e choix	Application de covoiturage
3e choix	Attribution de sujets

Mon compte | Mes résultats

Enregistrer mes choix

Exemple : interface administrateur

Attribution de sujets - Administration

Suivi des attributions Étape : premiers choix

Sujet	Attribués	En attente
Supervision réseau	2 / 2	3
Covoiturage	2 / 2	1

Paramètres | Stagiaires | Statistiques

Voir l'étape suivante